메모리풀(Mempool) 개요

개요

메모리풀(Mempool)은 CometBFT에서 트랜잭션을 임시로 저장하고 관리하는 핵심 컴포넌트입니다. 합의 과정에서 블록에 포함될 트랜잭션들을 수집하고, 검증하며, 다른 노드들과 공유하는 역할을 담당합니다. 메모리풀은 네트워크로부터 수신된 트랜잭션들을 ABCI 애플리케이션을 통해 검증한 후, 유효한 트랜잭션들만을 메모리에 보관합니다.

핵심 개념

트랜잭션 생명주기

1. **수신**: P2P 네트워크를 통해 트랜잭션 수신
2. **캐시 확인**: 이미 처리된 트랜잭션인지 캐시에서 확인
3. **사전 검증**: 크기 제한 등 기본적인 유효성 검사
4. **ABCI 검증**: 애플리케이션을 통한 트랜잭션 유효성 검증
5. **메모리풀 추가**: 검증된 트랜잭션을 메모리풀에 저장
6. **전파**: 다른 피어들에게 트랜잭션 전파
7. **블록 포함**: 합의 과정에서 블록에 포함
8. **제거**: 커밋된 트랜잭션을 메모리풀에서 제거

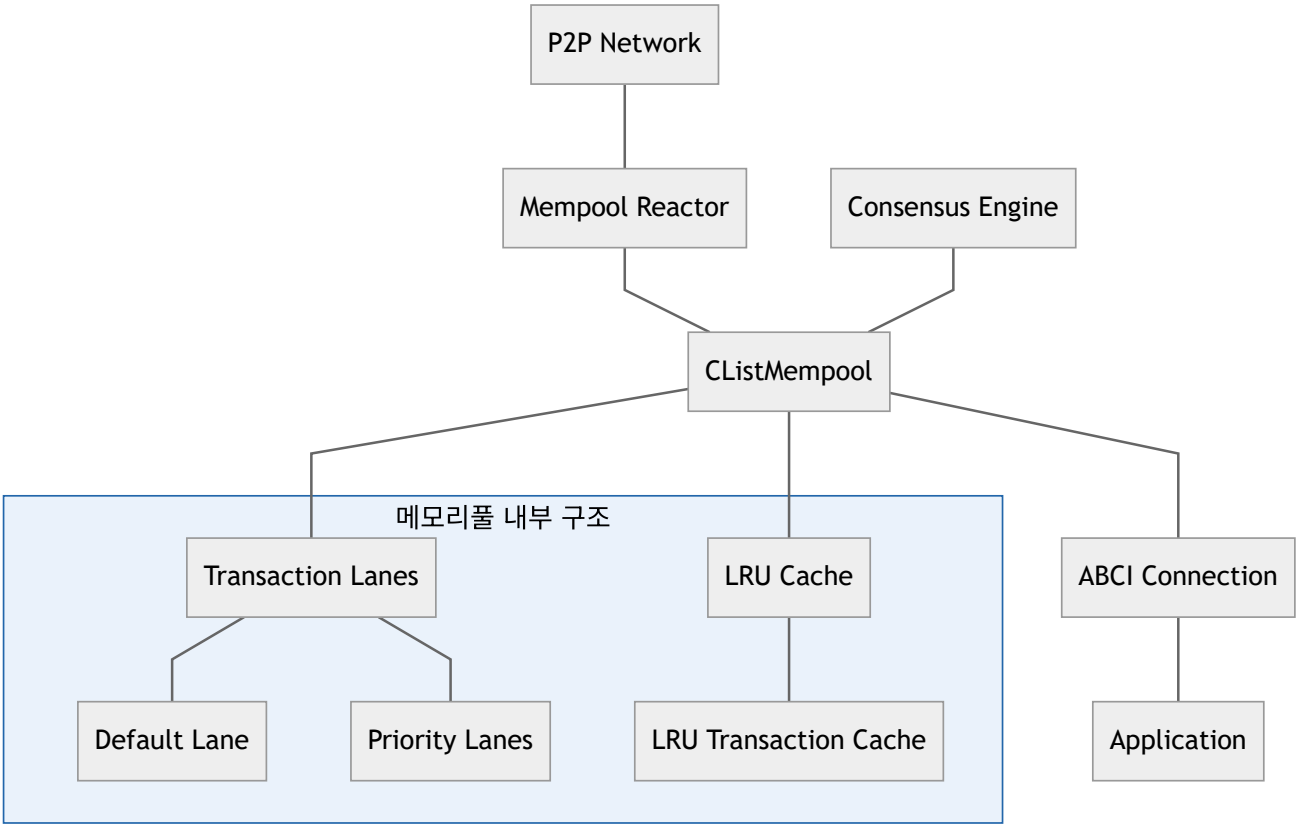
주요 용어

- **CheckTx**: 트랜잭션 유효성을 검증하는 ABCI 메서드
- **Recheck**: 새 블록 커밋 후 메모리풀 트랜잭션들을 재검증하는 과정

- **Lane**: 트랜잭션을 분류하고 우선순위를 관리하는 논리적 구분
- **Cache**: 중복 트랜잭션을 필터링하기 위한 LRU 캐시

아키텍처

전체 구조



핵심 컴포넌트

1. CListMempool

- **역할**: 메인 메모리풀 구현체
- **특징**: 동시성 안전한 연결 리스트 기반
- **기능**: 트랜잭션 저장, 검증, 관리

2. Transaction Lanes

- **목적**: 트랜잭션 분류 및 우선순위 관리
- **구조**: 각 레인은 독립적인 연결 리스트
- **장점**: 트랜잭션 타입별 차별화된 처리 가능

3. LRU Cache

- **기능**: 중복 트랜잭션 필터링
- **구현**: 해시맵과 연결 리스트 조합
- **효과**: 네트워크 대역폭 절약 및 성능 향상

동시성 처리

락킹 메커니즘

메모리풀은 여러 고루틴이 동시에 접근하는 환경에서 안전하게 동작해야 합니다:

1. **updateMtx (RWMutex)**

- 메모리풀 업데이트 시 배타적 락
- CheckTx와 Reap 작업 시 읽기 락
- 합의 엔진의 Update 호출 시 쓰기 락

2. **txsMtx (RWMutex)**

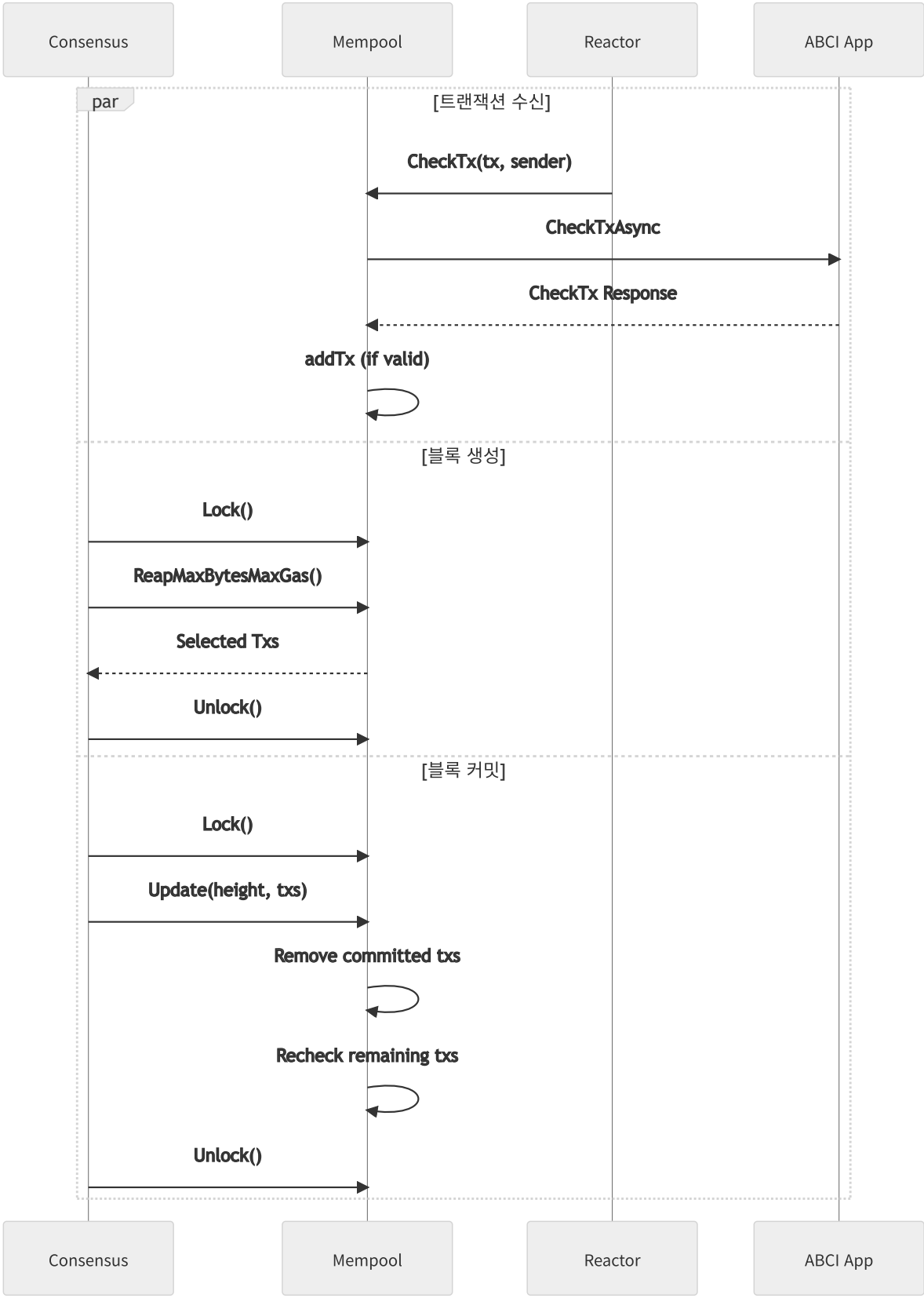
- 트랜잭션 맵과 레인 접근 시 사용
- 읽기 작업: Size, Contains, GetTxByHash
- 쓰기 작업: addTx, RemoveTxByKey

3. **addTxChMtx (RWMutex)**

- 새 트랜잭션 추가 알림 채널 보호

- 이터레이터 동기화에 사용

동시성 시나리오


Consensus
Mempool
Reactor
ABCI App
par
[트랜잭션 수신]
CheckTx(tx, sender)
CheckTxAsync
CheckTx Response
addTx (if valid)
[블록 생성]
Lock()
ReapMaxBytesMaxGas()
Selected Txs
Unlock()
[블록 커밋]
Lock()
Update(height, txs)
Remove committed txs
Recheck remaining txs
Unlock()
Consensus
Mempool
Reactor
ABCI App

데이터 구조

mempoolTx 구조체

```
type mempoolTx struct {
height  int64  // 트랜잭션이 검증된 블록 높이
gasWanted  int64  // 트랜잭션이 요구하는 가스량
tx types.Tx // 실제 트랜잭션 데이터
lane LaneID // 트랜잭션이 속한 레인
seq  int64  // 추가 순서 번호
timestamp time.Time // 생성 시간
senders sync.Map // 트랜잭션을 전송한 피어 목록
}
```

Lane 관리

```
type lane struct {
id LaneID // 레인 식별자
priority LanePriority // 우선순위 (높을수록 우선)
}
```

```
// 메모리풀 내부 레인 저장소
lanes  map[LaneID]*clist.CList // 각 레인의 트랜잭션 리스트
laneBytes  map[LaneID]int64  // 레인별 바이트 사용량
sortedLanes []lane  // 우선순위 순으로 정렬된 레인 목록
```

메모리 관리

크기 제한

- **Size**: 최대 트랜잭션 개수 (기본값: 5,000개)
- **MaxTxsBytes**: 전체 메모리풀 최대 크기 (기본값: 64MB)
- **MaxTxBytes**: 단일 트랜잭션 최대 크기 (기본값: 1MB)

레인별 분할

메모리풀 용량은 모든 레인에 균등하게 분할됩니다:

```
laneTxsCapacity := mem.config.Size / len(mem.sortedLanes)
laneBytesCapacity := mem.config.MaxTxsBytes / int64(len(mem.sortedLanes))
```

가비지 컬렉션

- **자동 제거**: 블록 커밋 시 포함된 트랜잭션 자동 제거
- **재검증**: 남은 트랜잭션들의 유효성 재확인
- **캐시 관리**: LRU 정책에 따른 캐시 엔트리 제거

성능 특성

시간 복잡도

- **트랜잭션 추가**: O(1) - 연결 리스트 끝에 추가
- **트랜잭션 검색**: O(1) - 해시맵 기반 인덱스
- **트랜잭션 제거**: O(1) - 직접 포인터 접근
- **캐시 조회**: O(1) - 해시맵 기반

공간 복잡도

- **메인 저장소**: O(n) - n은 트랜잭션 개수
- **인덱스**: O(n) - 트랜잭션 키 매핑
- **캐시**: O© - c는 캐시 크기

처리량 최적화

- **비동기 검증**: CheckTx 요청을 비동기로 처리
- **병렬 읽기**: 여러 고루틴이 동시에 읽기 작업 수행
- **효율적인 이터레이션**: 논블로킹 이터레이터 제공

참고 자료

- ABCI 인터페이스 문서
- P2P 네트워킹 문서
- 합의 프로세스 문서